

9 Automation

As the number of systems you manage increase, the amount of time you would spend (read waste) doing the same thing would increase exponentially, making the process of automating your Linux Administration task a necessity. Along side the major benefit of saving time, automation brings along advantages like consistency in your setup, regularity in mundane but critical tasks such as backup, compliance with standards and guidelines and the last but not the least, you get a homogeneous and easy to maintain setup. Likewise, you might think you don't need automation if you have only one server in your company or home. However, you might want it because backups and timely security updates are easy tasks for a busy system administrator to neglect, even in this most basic setup. Automation is already a core part of UNIX philosophy, and cron jobs have historically been the de facto method for automating UNIX tasks.

An important part of the automation is the ability to access all your systems from a single location and execute your commands safely, reliably and securely on it. For that reason, we will now introduce SSH, it is like a tool, what you do with it is completely up to you, reading the Appendix on “BASH Shell scripting” will prove immensely helpful.

Install the necessary tool by issuing the following command:

```
$ sudo apt-get install openssh-client openssh-server
```

To begin with, try logging into your own machine by issuing the following command:

```
$ ssh your_username@your_ip
```

Following this, you will be asked for your password and issued several warnings. Ignore them for now, we will come back to them. Don't fret.

You are now logged on to your own terminal via SSH, and you can do anything you want as if it were a native terminal. You can, of course, login to any remote machine via the same syntax.

```
$ ssh remote_machine_user_name@remote_machine_IP
```

So far so good, but till now your local and remote environment were isolated entities. To transfer the message between the matrix, use SCP, or secure copy.

```
$ scp local_source user@host:/remote_dest
```

```
$ scp userA@hostA:/remote_fileA userB@hostB:/remote_
```